A Task-Level Cost Model for LLM Endpoint Selection

with prompt caching, ordered fallbacks, time value, and process and parameter risk

Matthew Pollard

2026-09-04

Abstract

The price of a language-model call is not its list price. Over a multi-turn task the transcript is resubmitted at every turn, so the caller pays for a growing prefix N times. Whether that prefix is read from a prompt cache, rewritten into it, or processed cold; whether the request fails and is served by a fallback; and how long the caller waits all change the bill, and all of them differ between providers serving the same model. This document defines the cost of a *task* as the sum of its turn costs, derives its expectation in closed form under an explicit routing policy, separates process variance (realised misses) from parameter variance (uncertainty in the hit rate itself, which scales as N^4), and states how each input maps to a field OpenRouter publishes or to a caller assumption.

1 Setup

1.1 Task profile

A task is a sequence of N turns indexed $k = 1, \ldots, N$. At each turn the caller appends a new prompt tokens (user text or tool results) and the model emits o completion tokens, so the transcript grows by $d = a + o$ tokens per turn. At turn k the prompt is the full transcript so far: a *reusable prefix*

$$S_k = (k-1)\,d, \qquad k = 1, \ldots, N,$$

that was already sent on the previous turn, followed by the a new tokens, which no cache can have seen. The total prompt is $P_k = S_k + a$. Inter-turn gaps are assumed short relative to cache retention; Section 10 relaxes this.

1.2 Endpoint prices: three prompt prices, not two

An endpoint e bills prompt tokens in three categories, each with its own per-token price (USD per token; the tools display per million):

Symbol	Category	OpenRouter price field	Usage field
b_e	ordinary uncached input	`prompt`	—
r_e	cache read	`input_cache_read`	`cached_tokens`
w_e	cache write	`input_cache_write`	`cache_write_tokens`
c_e	completion	`completion`	`completion_tokens`
f_e	fixed fee per request (USD)	`request`	—

Providers differ in how they price writes: some charge a premium ($w_e = 1.25\,b_e$ or $2\,b_e$), some charge nothing beyond ordinary input ($w_e = b_e$), and some charge nothing at all for writes. When no write price is published we set $w_e = b_e$. When no read price is published the endpoint has no cache and $r_e = w_e = b_e$, $h_e = 0$. Prices from the endpoints API are already net of any advertised discount.

The billing primitive is a partition of the prompt into ordinary, read, and written tokens, $U_k + C_k + W_k = P_k$, with

$$I_k = f + b\,U_k + r\,C_k + w\,W_k. \tag{1}$$

Everything below is a policy for how (U_k, C_k, W_k) is generated.

1.3 Endpoint telemetry

OpenRouter publishes, per endpoint:

Symbol	Meaning	Window	Source
h_e^{obs}	cached prompt tokens / all prompt tokens	24 h	`cacheHitRate`
T_e	tokens served	24 h	`totalTokens`
u_e	fraction of successful requests	24 h	`uptime_last_1d`
ℓ_e^{50}, ℓ_e^{90}	time to first token, s	5 min rolling	`p50_latency`, `p90_latency`
g_e^{50}, g_e^{90}	output throughput, tokens/s	5 min rolling	`p50_throughput`, `p90_throughput`

Two percentile conventions are in play. OpenRouter's routing documentation defines a p90 threshold as a value that 90% of requests *beat*. The model-page statistics the tools read, however, are plain distribution percentiles: $\ell^{90} > \ell^{50}$ is the slow latency tail and $g^{90} > g^{50}$ is the *fast* throughput tail (for GLM 5.3 Flash on 2026-09-04, $g^{50} = 23.5$ and $g^{90} = 47$ tokens/s on one endpoint). The fits in Section 3 use the absolute log-ratio of the two quantiles, which is the same under either convention. The percentiles summarise a five-minute window while the rates summarise a day; the model treats both as current.

1.4 Caller parameters

Symbol	Meaning	Default
a	new prompt tokens per turn (user text, tool results)	2000
N	turns; default grows the context to 3×10^5 tokens	145
o	completion tokens per turn; default: 10^4 total output over N	69
v	value of the caller's time, USD per hour	5
γ	prefill speed as a multiple of decode throughput	100
τ_0	fixed per-request wait (network, queueing), seconds	0
ρ	routing policy: order or sticky	sticky (OpenRouter default)
$(e_1, \ldots, e_m)$	fallback chain; e_1 is the scored endpoint	(e_1, B), B a cold outside option
miss policy	prefix on a miss *rewritten* ($m = w$) or *processed* ($m = b$)	rewritten
cache mode	aggregate, cold, assumed, trace	aggregate
σ_h	epistemic standard deviation of the hit rate	0
$\lambda_{\text{proc}}, \lambda_{\text{par}}$	risk aversion to process and parameter risk	$0, 0$

2 Assumptions

Assumption 1 (Whole-prefix caching)**.** *On a successful turn the cached fraction $H_k \in [0, 1]$ of the reusable prefix is read at r. The remaining $(1 - H_k)S_k$ prefix tokens are billed at the miss price $m \in \{w, b\}$: $m = w$ if the provider rewrites the prefix into the cache on a miss (the default), $m = b$*

if it merely processes it uncached. The a new tokens are appended to the cache and billed at w. In the notation of (1)*: $C_k = H_k S_k$, $W_k = a + (1 - H_k)S_k$ under rewriting, or $W_k = a$, $U_k = (1 - H_k)S_k$ under processing.*

Assumption 2 (Independent cache fractions)**.** *The H_k are independent across turns with common mean $\mathbb{E}[H_k] = h$. The Bernoulli case $H_k \in \{0, 1\}$, $\Pr[H_k = 0] = 1 - h$, is the member of this family with the largest variance, $\mathrm{Var}[H_k] = h(1 - h)$; it is used only for bounds.*

Assumption 3 (Independent failures and an ordered fallback chain)**.** *A request to endpoint e fails with probability $q_e = 1 - u_e$, independently of everything else, and a failed request is not billed for tokens. A turn is attempted along the chain $e_1, e_2, \ldots, e_m$ until one succeeds. The last link is an outside option with $u_{e_m} = 1$ (or the task is declared failed with the residual probability). Fallback endpoints hold no cached prefix for this task unless the routing policy says otherwise.*

Assumption 4 (Time is linear in tokens)**.** *A request waits τ_0 seconds of fixed overhead, then prefills every prompt token the cache does not hold at $g_p = \gamma\, g$ tokens per second (roughly a hundred times decode), then generates o tokens at $s = 1/g$ seconds each. On a warm turn the uncached tokens are the a new ones; on a miss or a cold fallback they are $a + S_k$. Time is valued at v USD per hour; write $v' = v/3600$. The published time to first token is not charged: it is the prefill of whatever prompts the publisher's traffic contained, at that traffic's cache mix, and charging it on every turn alongside an explicit prefill model would double count and misprice warm turns. It is retained as a Pareto objective and for display.*

Assumption 5 (Missing telemetry is worst-case)**.** *If an endpoint lacks a latency, throughput, or uptime observation, the value is imputed as the worst observed among endpoints serving the same model. If a cacheable endpoint has no published hit rate it is scored cold, $h = 0$. An endpoint is never made cheaper by having less data.*

3 Inputs

3.1 Hit rate: the published number is a proxy with the wrong denominator

The published h^{obs} is cached tokens over *all* prompt tokens in the endpoint's 24-hour traffic. The model needs the cached fraction of the *reusable prefix*. Under Assumption 1 these differ by the share of new tokens in the prompt: for a turn with prefix S and a new tokens, cached tokens are HS and prompt tokens are $S + a$, so

$$h^{\mathrm{obs}} = h \cdot \frac{S}{S + a} \;\leq\; h.$$

The conversion factor depends on the *publisher's* traffic mix, which is not observable, so h^{obs} cannot be inverted exactly. It is a conservative proxy: it understates the reusable-prefix hit rate by the new-token share of whatever workloads that endpoint served. The tools therefore expose four cache modes: `aggregate` uses $h = h^{\mathrm{obs}}$ as-is; `cold` sets $h = 0$ for a bound; `assumed` takes a caller-supplied h; `trace` estimates h from the caller's own usage records, which report `cached_tokens` and `cache_write_tokens` per request.

No shrinkage toward a prior is applied. The published rate is an average over tens of thousands to millions of requests (the smallest primary GLM 5.3 Flash endpoint served 3×10^8 tokens on 2026-09-04, the largest 4×10^{10}), so its sampling error is below 0.3 percentage points for every endpoint. Any prior heavy enough to move it would penalise smaller providers for being smaller, not for being uncertain. The uncertainty that matters is epistemic: the window drifts day to day, and the workload mix is not the caller's. That enters as σ_h in Section 8.2.

3.2 Uptime

u is OpenRouter's request-weighted success fraction over 24 hours and is used as published. Token volume is not a sample size for uptime and is not used as one.

3.3 Latency and throughput: expected time from two quantiles

Expectations, not quantile blends, enter the cost. Fit a lognormal to each published pair. For time to first token (used for the frontier and display, not charged; Assumption 4), with median ℓ^{50} and 90th percentile ℓ^{90},

$$\sigma_\ell = \frac{\ln(\ell^{90}/\ell^{50})}{\Phi^{-1}(0.9)}, \qquad \mathbb{E}[\ell] = \ell^{50}\exp\big(\sigma_\ell^2/2\big), \qquad \Phi^{-1}(0.9) = 1.2816. \tag{2}$$

Generation time is linear in seconds per token $s = 1/g$, not in g, so the fit is on s. The published throughput quantile may be the fast or the slow tail depending on the source (Section 1.3); the lognormal is symmetric in $\ln s$, so either decile identifies σ_s:

$$\sigma_s = \frac{\big|\ln(g^{90}/g^{50})\big|}{\Phi^{-1}(0.9)}, \qquad \mathbb{E}[s] = \frac{1}{g^{50}}\exp\big(\sigma_s^2/2\big). \tag{3}$$

Latency is treated the same way, $\sigma_\ell = |\ln(\ell^{90}/\ell^{50})|/\Phi^{-1}(0.9)$. When only a median is published, $\sigma = 0$ and the median is used. Both expectations are increasing in the tail width, so a wider tail can only make an endpoint look worse.

4 Turn cost

Fix the scored endpoint $A = e_1$ with prices b, r, w, c, f, miss price m, inputs $h, u, \mathbb{E}[s]$, $g_p = \gamma g^{50}$, and $q = 1 - u$. Write $\rho = v'/g_p$ for the time value of prefilling one prompt token.

4.1 Successful turn

$$X_k^{\text{ok}} = \underbrace{f + w\,a + c\,o}_{\text{fixed}} + \underbrace{v'\,\tau_0 + \rho\,a + v'\,o\,s}_{\text{overhead, new-token prefill, decode}} + \underbrace{S_k\,\big[\,m - H_k\,(m - r)\,\big]}_{\text{prefix money}} + \underbrace{\rho\,(1 - H_k)\,S_k}_{\text{prefix re-prefill}}. \tag{4}$$

Taking expectations,

$$\mu_k^{\text{ok}} = \alpha + \pi\,S_k, \qquad \alpha = f + w\,a + c\,o + v'\,\tau_0 + \rho\,a + v'\,o\,\mathbb{E}[s], \qquad \pi = h\,r + (1 - h)\,(m + \rho). \tag{5}$$

π is the *effective prefix price*, in dollars of money and time per prefix token. The new tokens never receive it. A cache miss therefore costs both the price spread $m - r$ and ρ per token of waiting; on a 200k-token prefix at $g_p = 3000$ tokens/s the wait alone is over a minute.

4.2 Failed turn on a two-link chain

Take the chain (A, B) with $u_B = 1$; the general chain is in Section 6. With probability q the request to A fails. The caller has waited the overhead τ_0 for nothing, then the turn is served cold by B: every prompt token is new to B, written at w_B and prefilled at $g_{p,B}$.

$$\begin{aligned} X_k^{\text{fail}} = v'\tau_0 + \underbrace{f_B + w_B\,(S_k + a) + c_B\,o + v'\,\tau_0 + \rho_B\,(S_k + a) + v'\,o\,\mathbb{E}[s_B]}_{\text{cold turn on } B} \\ + \underbrace{\mathbf{1}[\mathsf{order}]\;\mathbf{1}[k < N]\;h\,(m - r + \rho)\,d}_{\text{return penalty}}. \end{aligned} \tag{6}$$

This branch is deterministic given the failure, so $\mu_k^{\text{fail}} = \beta + (w_B + \rho_B)S_k + \mathbf{1}[\mathsf{order}]\mathbf{1}[k < N]\, h(m - r + \rho)d$ with $\beta = 2v'\tau_0 + f_B + w_B a + c_B o + \rho_B a + v' o \mathbb{E}[s_B]$.

The return penalty depends on the routing policy. Under an explicit provider order, OpenRouter disables load balancing and the next turn goes back to A. A never saw turn k's completion, so of S_{k+1} only S_k can be cached and the extra d tokens are written at w rather than read at the blended price, and re-prefilled; the expected extra cost is $h(m-r+\rho)d$, booked here at turn k (there is no turn $N+1$, hence the indicator). Under sticky routing, OpenRouter's default, the provider that served the last cached request becomes the sticky provider, so after a fallback the task *stays on* B: there is no return penalty, but every subsequent turn is billed at B's prices and hit rate. That is a regime change, not a per-turn surcharge, and it has its own closed form in Section 7.

4.3 Expected turn cost under order

$$\begin{aligned}\mathbb{E}[X_k] &= (1-q)\,\mu_k^{\text{ok}} + q\,\mu_k^{\text{fail}} \\ &= \underbrace{(1-q)\,\alpha + q\,\beta}_{A_0} + \underbrace{\big[(1-q)\,\pi + q\,(w_B + \rho_B)\big]}_{\Pi} S_k + q\,\mathbf{1}[k < N]\,h\,(m - r + \rho)\,d. \end{aligned} \tag{7}$$

Apart from the last-turn indicator the turn cost is affine in the prefix, with intercept A_0 and slope Π, the failure-adjusted effective prefix price.

5 Task cost

Proposition 1 (Task cost under order)**.** *The task cost is the sum of the turn costs, $X_{task} = \sum_{k=1}^{N} X_k$. Under Assumptions 1–4, the two-link chain, routing policy order, and constant growth $S_k = (k-1)d$,*

$$\boxed{\mathbb{E}[X_{task}] = N\,A_0 \;+\; \Pi\,d\,\frac{N(N-1)}{2} \;+\; q\,h\,(m - r + \rho)\,d\,(N-1)} \tag{8}$$

Proof. Sum (7) over k using $\sum_{k=1}^{N}(k-1) = N(N-1)/2$ and $\sum_{k=1}^{N}\mathbf{1}[k<N] = N-1$. □

5.1 Decomposition

Expanding Π separates what the caller pays under a perfect cache from what misses and failures add:

$$\begin{aligned}\Pi\,d\,\frac{N(N-1)}{2} = \;& \underbrace{r\,d\,\tfrac{N(N-1)}{2}}_{\text{cached-read baseline}} + \underbrace{(1-q)(1-h)(m-r+\rho)\,d\,\tfrac{N(N-1)}{2}}_{\text{miss premium}} \\ & + \underbrace{q\,(w_B + \rho_B - r)\,d\,\tfrac{N(N-1)}{2}}_{\text{failure premium}}.\end{aligned} \tag{9}$$

All three are $\Theta(N^2)$: there are N submissions and the k-th exposes a prefix of $(k-1)d$ tokens, so every per-token surcharge on the prefix is paid on a quadratically growing volume. A hit rate of 95% is not a 5% surcharge on a long task; the miss premium is $\Theta((1-h)N^2)$ while NA_0 does not grow with context at all. The return-penalty term is linear in N and $O(q)$.

Remark 1 (Mean-prefix equivalence)**.** *Ignoring the $O(q)$ return term,* (7) *is affine in S_k, so $\mathbb{E}[X_{task}] = N \cdot \mathbb{E}[X_k \mid S_k = \bar{S}]$ with $\bar{S} = d(N-1)/2$. The task equals N representative turns at the mean prefix, not one. The factor N does not affect the ranking of endpoints but does affect totals.*

Remark 2 (Relation to the single-turn scorer)**.** *The current scorer takes one prompt size C and applies π to all of it. That treats the new tokens as partly cached and underprices each turn by $h\,(m-r)\,a$; it uses two prompt prices where there are three; it charges no time for re-prefilling a missed prefix; and it reports one turn, not N. Replacing C by the pair (a, S_k), adding the write price, and summing over k recovers* (8)*.*

5.2 Comparing endpoints

For endpoints A and A' serving the same model,

$$\mathbb{E}[X^{A}_{\text{task}}] - \mathbb{E}[X^{A'}_{\text{task}}] = N\left(A_0 - A'_0\right) + \left(\Pi - \Pi'\right) d\,\frac{N(N-1)}{2} + O(q)\,(N-1). \tag{10}$$

$A_0 - A'_0$ contains price differences, time-value differences, and $O(q)$ failure terms, so it does not vanish in general. When the two endpoints have identical prices and $v = 0$ it is $O(q)$, and the gap is then dominated by the quadratic term, which is driven by h and u.

6 General fallback chain

For the chain $e_1, \ldots, e_m$, let $R_j = \prod_{i<j} q_{e_i}$ be the probability that attempt j is reached. Each reached attempt succeeds with probability u_{e_j} at cost $\mu^{S}_{j,k}$ (warm on e_1, cold on any e_j, $j \geq 2$) or fails at cost $\mu^{F}_{j,k} = v'\tau_0$ (the wasted wait). Then

$$\mathbb{E}[X_k] = \sum_{j=1}^{m} R_j \Big[u_{e_j}\,\mu^{S}_{j,k} + (1-u_{e_j})\,\mu^{F}_{j,k}\Big] + R_{m+1}\,\Xi, \tag{11}$$

where R_{m+1} is the probability every link fails and Ξ is the cost assigned to a failed task (or $R_{m+1} = 0$ if $u_{e_m} = 1$). The two-link formula (7) is the case $m = 2$, $u_B = 1$; using a fallible B instead changes the expectation by $O(q_A q_B)$. Setting $B = A$ is a proxy for "retried later, cold" and should not be read as the same endpoint succeeding with certainty after failing.

7 Sticky routing: failures migrate the task

Under sticky routing with two endpoints, the task runs warm on A until A's first failure, is served cold by B on that turn, and then runs warm on B for the rest of the task. Let $x = 1 - q$. The probability the task is still on A at turn k is x^{k-1}, and

$$\mathbb{E}[X_{\text{task}}] = \sum_{k=1}^{N} x^{k-1}\Big[x\,(\alpha + \pi S_k) + q\,(\beta + w_B S_k)\Big] + \sum_{k=1}^{N}\left(1 - x^{k-1}\right)\left(\alpha_B + \pi_B S_k\right), \tag{12}$$

where α_B, π_B are B's warm intercept and effective prefix price. With $G_0(x) = \sum_{k=1}^{N} x^{k-1} = (1-x^N)/(1-x)$ and $G_1(x) = \sum_{k=1}^{N}(k-1)x^{k-1} = x\big(1 - Nx^{N-1} + (N-1)x^N\big)/(1-x)^2$,

$$\mathbb{E}[X_{\text{task}}] = \big(x\alpha + q\beta - \alpha_B\big)G_0(x) + \big(x\pi + q w_B - \pi_B\big)\,d\,G_1(x) + N\alpha_B + \pi_B\,d\,\frac{N(N-1)}{2}.$$

The probability the task ends on B is $1-(1-q)^N$. For $N=200$ this is 63% at $u=0.995$ and 89% at $u=0.989$: under the default routing policy a long task on any endpoint short of near-perfect uptime will more likely than not finish on its fallback, and it is the fallback's hit rate and prices that govern most of the quadratic term. Ranking endpoints by their own h alone is therefore only valid under order, or when $qN \ll 1$.

8 Variance and risk

Two distinct kinds of variance apply, and they scale differently in N.

8.1 Process variance: realised misses and failures

Under Assumptions 2 and 3 with routing order, X_k depends only on (H_k, F_k) where F_k is the failure indicator, so the turns are independent and $\mathrm{Var}[X_{\text{task}}] = \sum_k \mathrm{Var}[X_k]$. By the law of total variance,

$$\mathrm{Var}[X_k] = (1-q)\, V_k^{\text{ok}} + q\, V_k^{\text{fail}} + q(1-q)\left(\mu_k^{\text{fail}} - \mu_k^{\text{ok}}\right)^2, \tag{13}$$

with $V_k^{\text{fail}} = 0$ (the cold branch is deterministic) and

$$V_k^{\text{ok}} = (m-r+\rho)^2\, S_k^2\, \mathrm{Var}[H_k] \;\le\; (m-r+\rho)^2\, h(1-h)\, S_k^2. \tag{14}$$

Summing,

$$\begin{aligned}\mathrm{Var}[X_{\text{task}}] \;\le\; & (1-q)\,(m-r+\rho)^2\, h(1-h)\, d^2\, \frac{(N-1)N(2N-1)}{6} \\ & + q(1-q) \sum_{k=1}^{N} \left[(\beta-\alpha) + (w_B + \rho_B - \pi) S_k + \mathbf{1}[k<N]\, h\,(m-r+\rho)\, d\right]^2.\end{aligned} \tag{15}$$

The first term is attained by the Bernoulli model; its standard deviation grows as $N^{3/2}$. Because the published h identifies only $\mathbb{E}[H_k]$, this is an upper bound on process risk, not an estimate of it, and no normal-tail statement should be made from it.

Remark 3 (Bookkeeping of the return penalty)**.** *Equation* (6) *books the return penalty at turn k as an expected quantity. If instead the realised extra cost is attributed to turn $k+1$, X_{k+1} depends on F_k and the turns are no longer independent;* $\mathrm{Cov}(X_k, X_{k+1}) = O(q)\,(m-r+\rho)\, d\, S_{k+1}$ *terms appear. The expectation is unchanged either way. The model as written is exact for its own bookkeeping and neglects covariance of order q under the alternative.*

8.2 Parameter variance: the hit rate itself is uncertain

Suppose the true reusable-prefix hit rate for this task is $h+\delta$, with $\mathbb{E}[\delta] = 0$ and $\mathrm{Var}[\delta] = \sigma_h^2$, shared across all turns (drift of the 24-hour window, or mismatch between the published workload and the caller's). Since $\partial \mathbb{E}[X_{\text{task}}]/\partial h = -(1-q)(m-r+\rho)\sum_k S_k + O(q)$,

$$\sigma_{\text{par}} = (1-q)\,(m-r+\rho)\,\sigma_h\, d\, \frac{N(N-1)}{2} = O(N^2), \qquad \mathrm{Var}_{\text{par}} = O(N^4). \tag{16}$$

A shared error in h is paid on every prefix token of every turn, so its standard deviation grows as N^2 and eventually dominates the $N^{3/2}$ process term. For a task long enough that cache economics matter, being wrong about h by a few points is a larger risk than any realised run of misses. σ_h is a caller input: the observed day-to-day standard deviation of the published rate once history is recorded, or a stated band for workload mismatch. It is not derived from token volume.

8.3 Decision rule

Given a quality constraint $Q_m \geq q_{\min}$ on the model m served, choose

$$\min_{(m,e)} \; J(m,e) = \mathbb{E}[X_{\text{task}}] + \lambda_{\text{proc}}\,\sigma_{\text{proc}} + \lambda_{\text{par}}\,\sigma_{\text{par}} \quad \text{s.t.} \quad Q_m \geq q_{\min}, \tag{17}$$

with both λ zero for a risk-neutral caller. λ_{proc} prices operational risk (a bad run of misses); λ_{par} prices epistemic risk (the input was wrong). The Pareto tools report the non-dominated set over

$$\big(\mathbb{E}[X_{\text{task}}]\downarrow,\ \mathbb{E}[\ell]\downarrow,\ g^{50}\uparrow,\ h\uparrow,\ u\uparrow\big)$$

so that a caller who will not commit to v and the λs can still see the trade-offs.

9 Worked example

GLM 5.3 Flash, primary endpoints, 2026-09-04. Every endpoint lists $b = 0.075$, $r = 0.015$, $c = 0.25$ USD per million tokens and publishes no write price, so $w = m = b$ and only telemetry separates them. Task profile $a = 2000$, $o = 500$, $N = 200$, $v = 0$ (so $\rho = 0$), routing order, chain $(A, A$ cold).

Endpoint	h	u	Π (\$/M)	$\mathbb{E}[X_{\text{task}}]$
Z.ai	0.885	0.989	0.0225	\$1.17
DeepInfra	0.469	0.995	0.0470	\$2.39

With $\sum_k S_k = d\,N(N-1)/2 = 49.75$M prefix tokens, the quadratic term is \$1.12 on Z.ai and \$2.34 on DeepInfra, against a linear term of \$0.055 on both and a return-penalty term below \$0.001. At the default single turn of the current tools the two endpoints differ by 29%; over the task they differ by 104%, almost all of it from the hit rate, with uptime moving Π in the third decimal. The process bound (15) gives standard deviations of \$0.081 and \$0.122; the bound on the standard deviation of their difference is $\sqrt{0.081^2 + 0.122^2} = \$0.146$, so the \$1.22 gap is about 8.3 such units. Since these are bounds, that is a statement about separation, not a tail probability. With $\sigma_h = 0.03$ (three points of drift), the parameter standard deviation is \$0.09 on either endpoint at $N = 200$ (it does not depend on h), already comparable to the process bound; at $N = 800$ it is \$1.44 against a process bound of \$0.63, and the ratio keeps growing as $\sqrt{N}$.

Under sticky routing the picture changes: a 200-turn task starting on Z.ai has an 89% chance of finishing on its fallback, and its expected cost is mostly the fallback's. The cheaper endpoint by hit rate is also the one more likely to be abandoned.

10 Beyond independent misses

Assumption 2 is the weakest link. The refinements are explicit knobs because OpenRouter's data cannot identify them.

Eviction. Caches expire after a time-to-live T_{ttl}. With inter-turn gap τ_k, replace the constant hit rate by $h_k = h \cdot \mathbf{1}[\tau_k < T_{\text{ttl}}]$ or $h_k = h\,e^{-\tau_k/T_{\text{ttl}}}$. Tool-call bursts then hit and human think-time misses.

Minimum cacheable length and cache cap. Providers publish a minimum prompt length below which nothing is cached (1,024 to 4,096 tokens for the major closed models), so $H_k = 0$ for P_k below the threshold. If a provider caches at most L tokens the cached part of the prefix is $\min(S_k, L)$ and the remainder is billed at m. Whether exceeding the cap instead invalidates the whole prefix is provider-specific and should be a switch, not an assumption.

Provider state. The general object is a serving endpoint Z_k and a cache-state vector $\mathcal{C}_k = (C_{e,k})_e$ giving the longest prefix each endpoint holds and when it was last touched. A transition updates only the cache of the endpoint that successfully served the turn. Turn k hits on Z_k only if $C_{Z_k,k}$ covers the current prefix within its TTL. order and sticky are the two policies for Z_k that have closed forms above; session-keyed stickiness and load-balanced routing without an order are further policies on the same state space. The independent model is the memoryless approximation.

Compounding. If a disruption at turn t_j leaves the task cold for R_j turns, the premium is $(m - r) \sum_j \sum_{k=t_j}^{t_j+R_j-1} S_k$. With fixed R_j this is still $O(N^2)$; if recovery scales with task length it is $O(N^3)$; sticky migration is the case $R_j = N - t_j$ with the fallback's prices substituted.

11 Reporting and implementation

The model is implemented in `openrouter_frontier/scoring.py`: the expectation, decomposition, bounds, and both routing policies are evaluated turn by turn (the loop is exact and needs no geometric-sum special cases), and the test suite checks the implementation against the closed forms (8) and (12) and the worked example above. The default profile is the one in Section 1.4.

The unit of optimisation is the task. For each endpoint the tools should report

- $\mathbb{E}[X_{\text{task}}]$ in dollars per completed task under the stated routing policy, with the decomposition (9): fixed, time, cached-read baseline, miss premium, failure premium, return penalty;
- the perfect-cache cost ($h = 1$) and cold-cache cost ($h = 0$) as bounds, and under sticky the probability of migrating to the fallback;
- the process bound σ_{proc}, the parameter term σ_{par}, and J;
- total ordinary, cache-read, cache-write, and completion tokens;
- the inputs $h, u, \mathbb{E}[s], g_p$ (and the published $\mathbb{E}[\ell]$ for reference), the cache mode and miss policy used, and the timestamps of the windows they summarise.

Per-million normalisation is secondary: repeatedly submitted transcript tokens make the denominator ambiguous, and the figure falls with context while the bill rises.